

In any application there are thousands of classes.

- Managing those classes Difficult
- There are some rules to manage multiple classes.

Otherwise :-

- Bugs will be introduced

So we have to write such code - Cleaner, understandable, maintain an particular architecture

If your code is good written then :-

- It is easy to find Bugs.
- — " — to introduce new feature

What happens if we don't follow design rules:-

1. Maintainability :-

- Easily introduce new feature
- less bugs introduce while adding new feature

2. Readability

- Easy to understand by other

3. Debugging

- less time it should take to solve any bug.

Problem

SOLID

S: Single Responsibility Principle (SRP)

O: Open - class principle (OCP)

L: Liskov substitution principle (LSP)

I: Interface Segregation principle (ISP)

D: Dependency Inversion principle (DIP)

So: Single Responsibility Principle

* A class should have only one reason to change

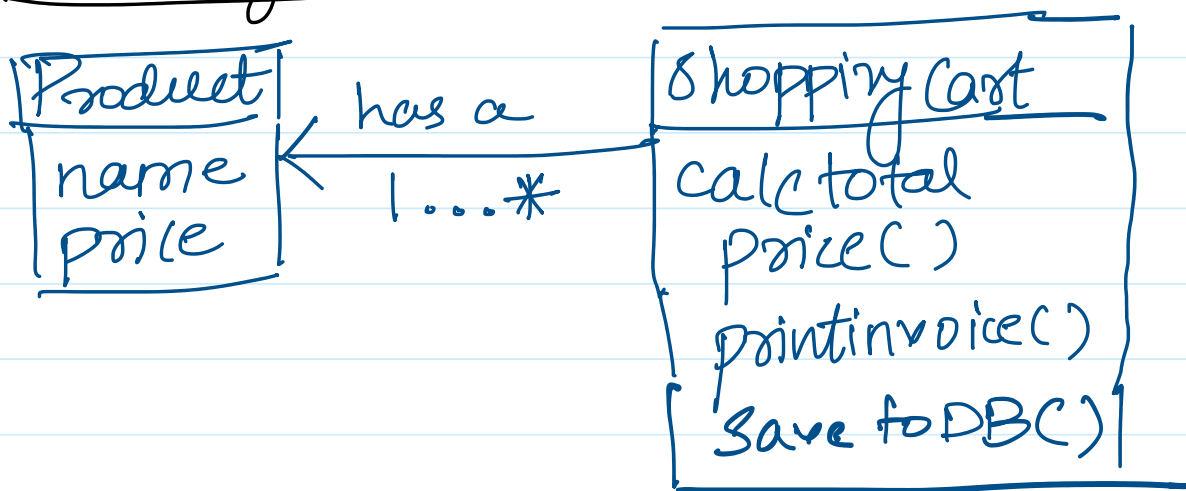
* A class should do only one thing.

एक class → एक काम करे → It should do only one thing

NO Multiple responsibility

All attributes and method in that class must handle only one responsibility

UML Diagram:-



Here you can see shoppingcart is doing multiple things.

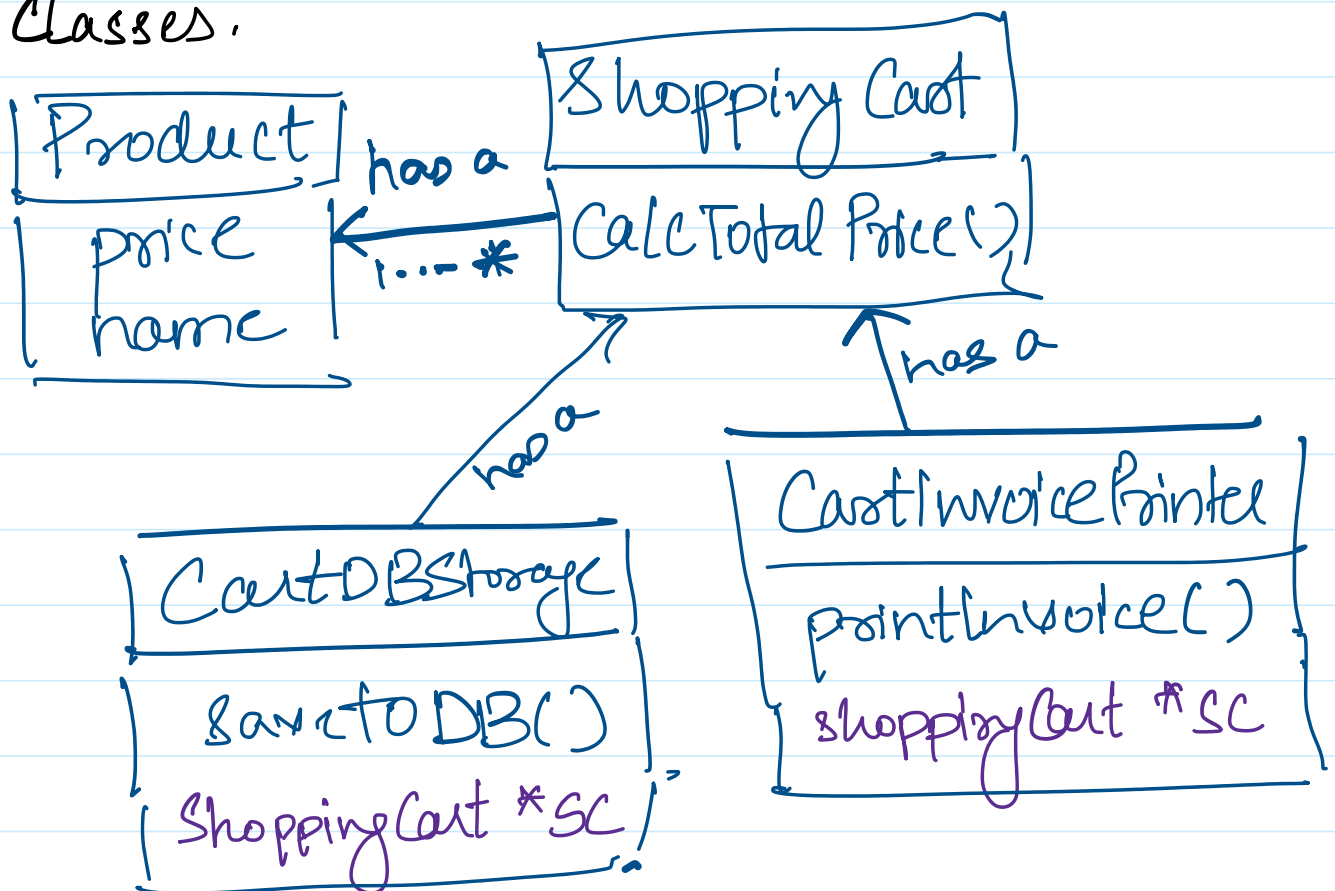
multiple things.

- so, when you want to change the logic of
save → change "saveToDB()
print → change "printInvoice()
total Calc. → change "calcTotalPrice()

So, here for multiple reason we are changing the same class "ShoppingCart"

So, SRP is failing

so, to follow SRP we use composition
Let's Break "ShoppingCart" in multiple
Classes.



Both CartDBStorage & Cart InvoicePrinter
will have reference of ShoppingCart

will have reference of ShoppingCart

Now every class is holding only one responsibility

Now, SRP followed ✓

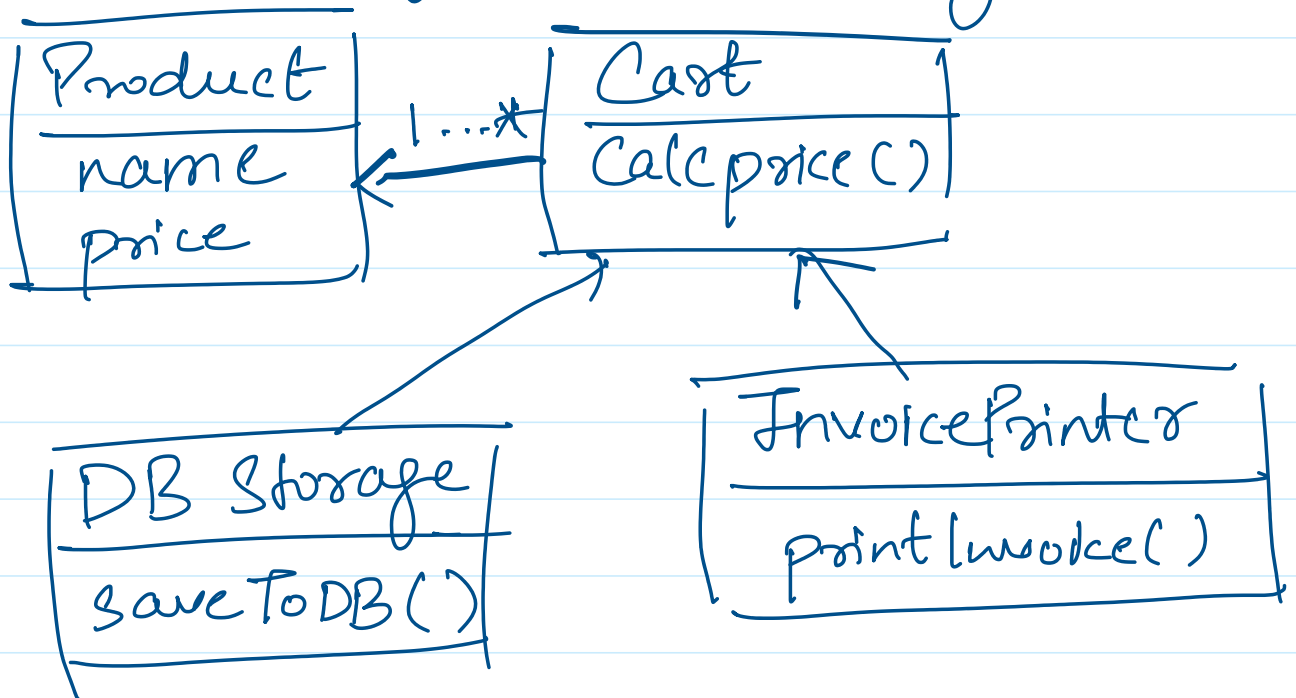
Note: Code.... See Code in notes.

SRP doesn't means each class should have only one method. They can have multiple method but all should be responsible for doing same thing.

O : Open - close principal

A class should be open for extension but close for modification.

When you introduce new feature don't change the existing code.



Previously, saveToDB() was only for SQL DB, now I want to do it for MongoDB and also want to save it in some file. :-

1st Approach

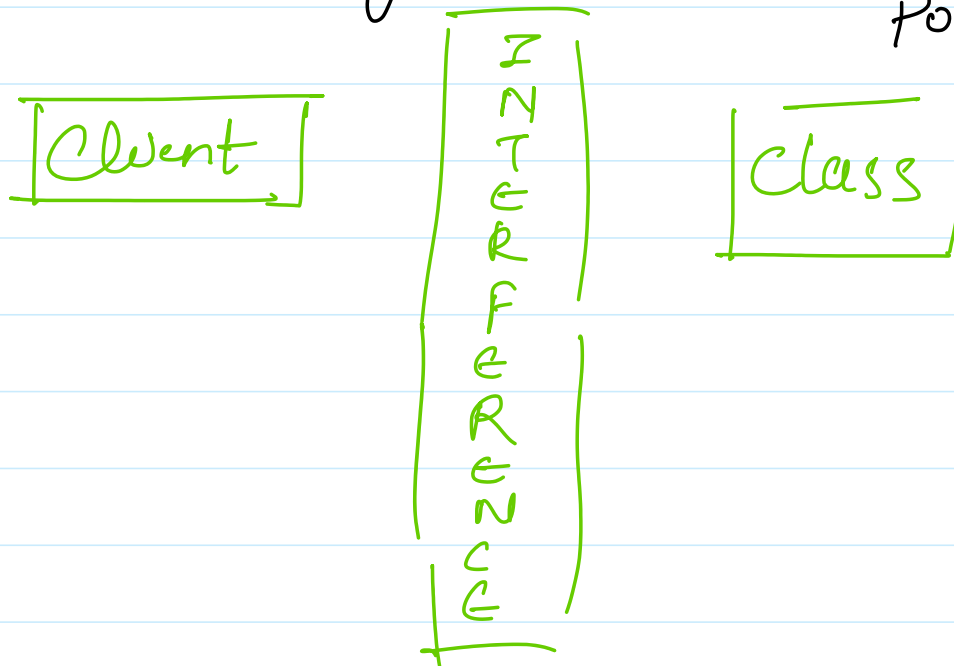
Introduce 2 new method in DB Storage Class i.e. saveToMongo() & saveToFile()

We broke OCP X, we are changing

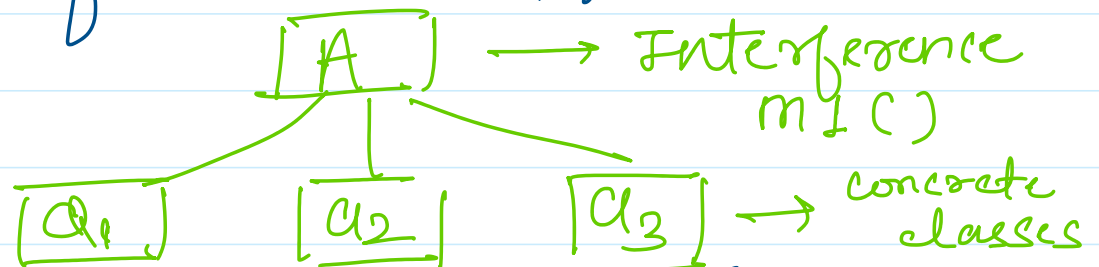
we broke OCP X, we are changing the existing class.

so solution is - Abstraction
- Inheritance
- Polymorphism

Solution using Abstraction, Inheritance, Polymorphism



Interference hide the actual implementation from the Client



This actually implement the abstract method m1()

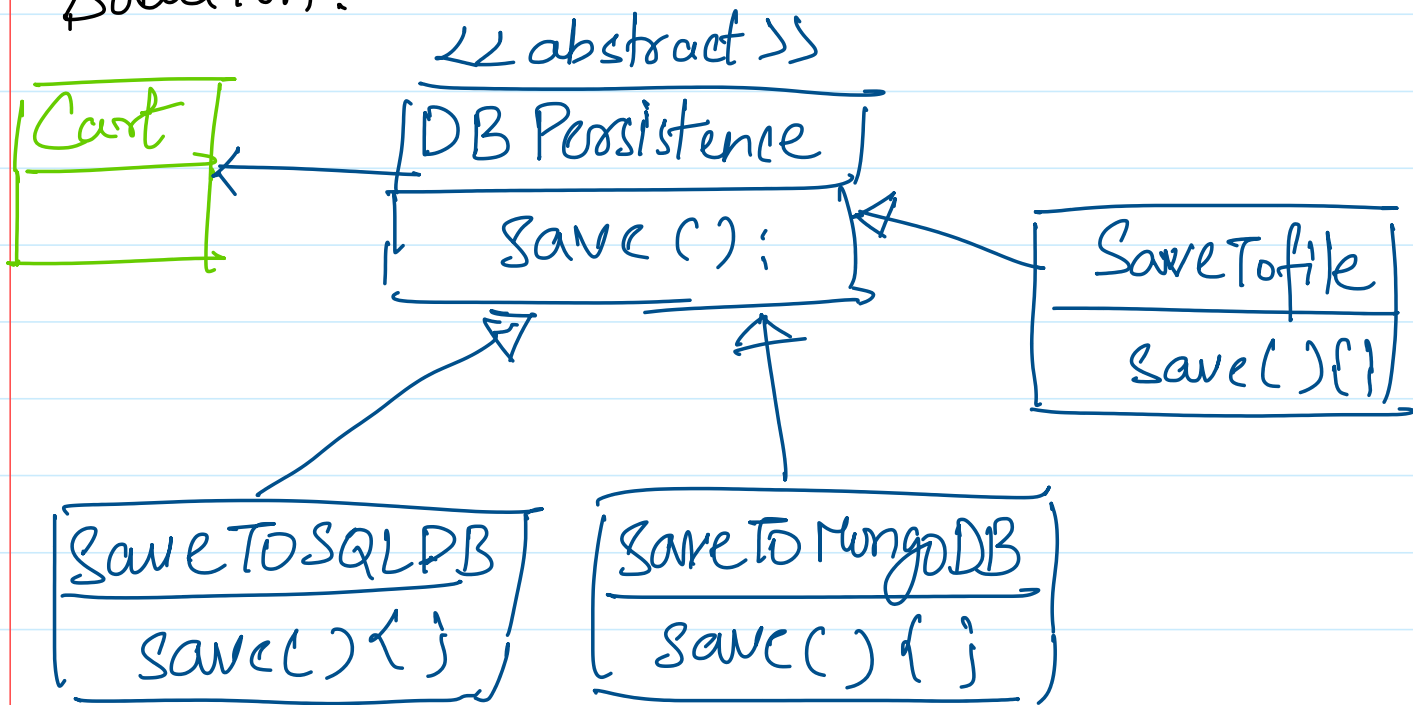
A a = new a1() } Method

A a_2 $new\ a_1()$
 $new\ a_2()$
 $new\ a_3()$

Method
Overriding

a_1, a_2, a_3 classes implements the
 $m_1()$ method of A — Inheritance

Solution:-



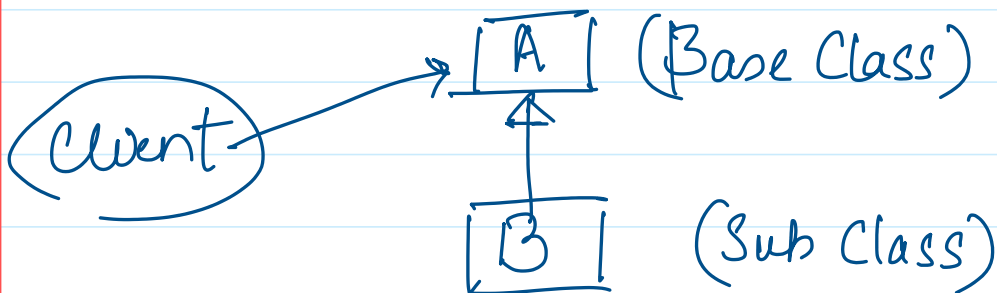
Now in future → if, we have to save
 in Cassandra DB then again we
 will make new class.

Code → see Notes...

OCP ✓ followed

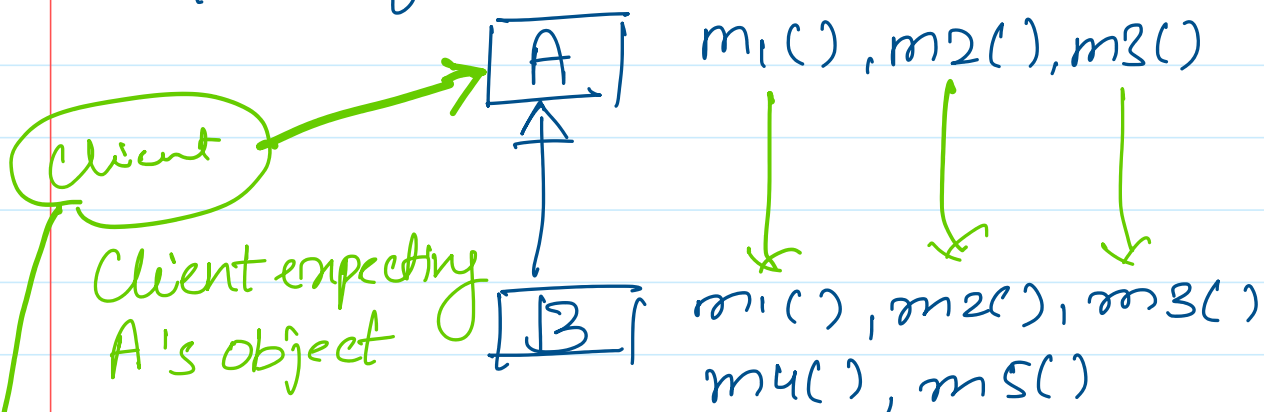
L: Liskov Substitution Principle.

Subclasses should be substitutable for their Base Classes.



Any Client who is expecting a reference of 'A' in any method. Even if we pass 'B' reference then also there should be no code breakage.

Means, B class should be substitutable in place of A.



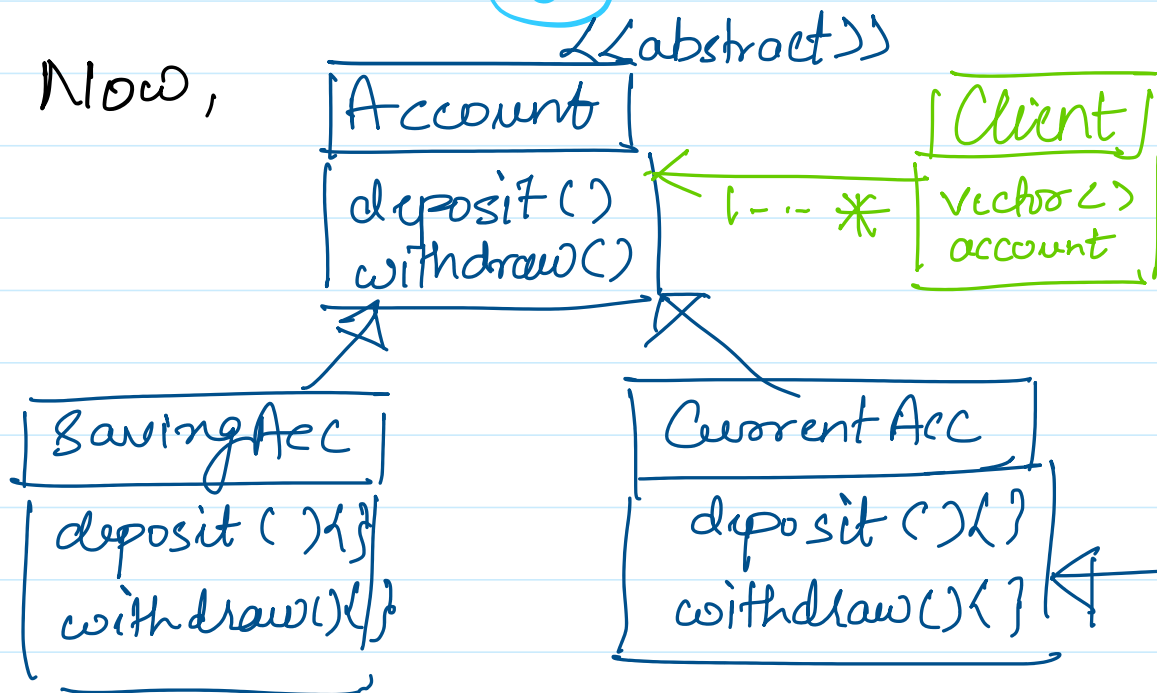
'B' always expand the 'A' class but don't narrow down 'A' - LSP principle

→ randomMethod(A * a) ✓

$\rightarrow$ randomMethod($A * a$) {
 $a \rightarrow m1()$
 $a \rightarrow m2()$
 $a \rightarrow m3()$
 }

if we pass B's obj
 $A * a = \text{new } B()$
 $a \rightarrow m1()$
 $a \rightarrow m2()$
 $a \rightarrow m3()$

$\rightarrow$ Even if we pass B's object then also it can call $m1()$, $m2()$, $m3()$
 No Problem 😊

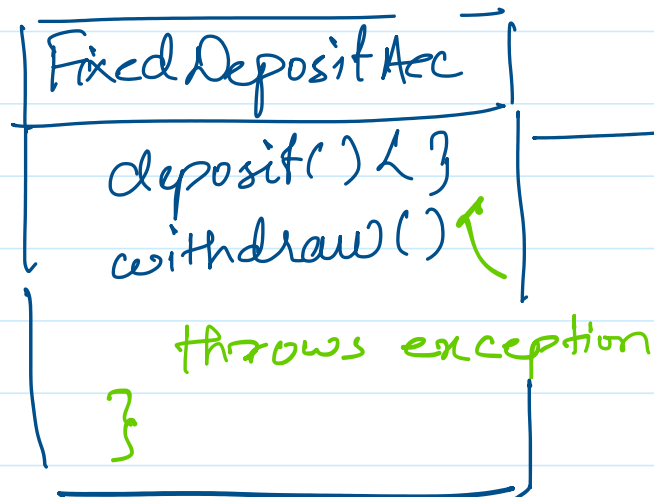


Suppose, client has List of accounts and he is able to call deposit() & withdrawal() on those accounts easily

Now How LSP breaks:

Suppose new type of account comes "Fixed Deposit Acc" - In this only deposit()

suppose new type of account called
 "Fixed Deposit Acc" - In this only deposit()
 can be called but not withdrawal().
 So, we make new child class of "Account"



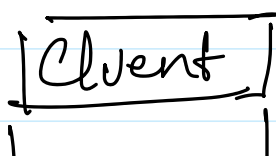
To restrict user from calling
 withdraw() we throw exception
 in withdraw()

But Client is unaware of it and
 when it pass object of "FixedDepositAcc"
 in withdraw() then it throws exception
 and code breaks.

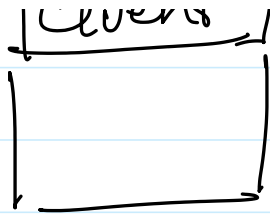
So, "FixedDepositAcc" class is not
 'substituable' for his parent class
 "Account"

To solve this:-

1st Approach:-



for (acc: account) {
 if (acc → type of
 "FixedDepositAcc")

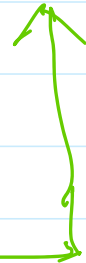


```

if (acc → type of
    (fixedDeposit))
{
    only call
    deposit()
    not withdraw()
}

```

we know client must be looping through all clients, so, put condition like this



BAD approach

→ b/c now our client is tightly coupled, it should only talk to Interface. client should not know about types of account.

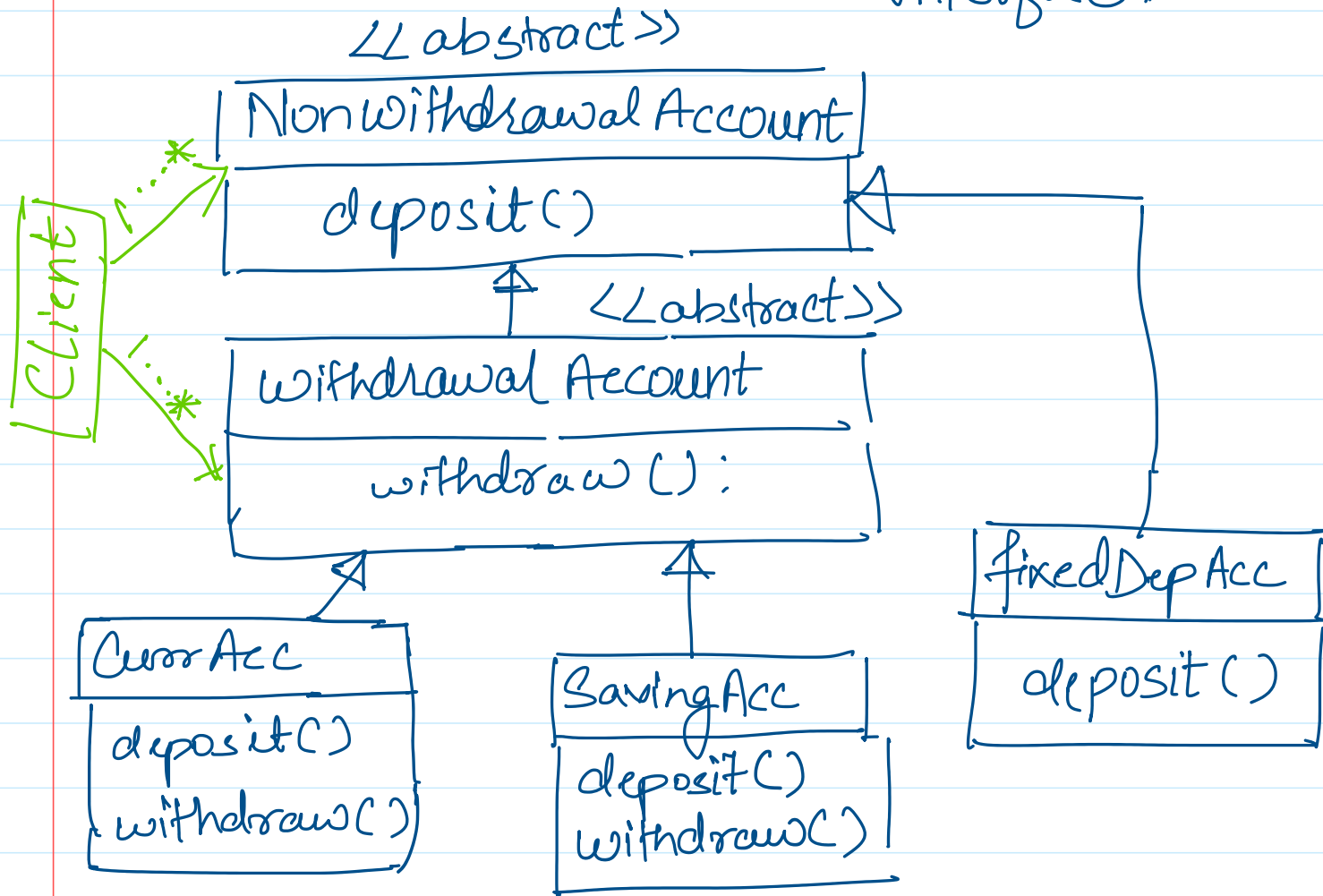
And in future suppose new type of account comes which have some condition then again we go and change the logic of Client. So we are also breaking OCP.

2nd Approach :-

"fixedDeposit Acc" should not be the child of "Account" class b/c it is narrowing down its functionality.

Let's make new parent class/
Interface

acc → make new person class/
Interface.



Now it follows LSP ✓

Now Client knows it has 2 Interface and it talk to them only.

Code → See Notes...